

Design + Validation and Verification Document

VERSION 1.2, LAST UPDATED 4/7/2026

INCLUDES UNIT TEST RESULTS

Fall 2025 – Winter 2026 Computer Science Capstone

Battery SOC Algorithm Evaluation Multitool

Group 12

Dason Wang
Aidan McLean
Paarth Kadakia
Ellen Xiong

Table of Contents

1. [Controls / Versions](#)
 - 1.1 Definitions and Prerequisite Knowledge
2. [Purpose Statement](#)
3. [Component Diagrams](#)
4. [Component Behaviours](#)

(Includes parts 5-6 on the project outline, *relationship-requirements between components*)

 - 4.1 [Frontend](#)
 - 4.2 [Backend](#)
5. [User Interface](#)
6. [V&V Component Test Plan](#)
7. [Appendix](#) (additional four pages not part of core document)

1. Controls / Versions

Date	Version	Developers	Change
January 23, 2026	1.0	Entire Team	Initial Design Doc

1.1 Definitions and Prerequisite Knowledge:

- SOC State of Charge (of a battery)
- Dask Python Library for Parallelization, orchestrates data through **Dataframes**
- Coiled Third Party Service for distributed compute management, VM instancing
- RMSE Root Mean Square Error, measures average magnitude of errors

2. Purpose Statement

Battery State of Charge (SOC) estimation requires specialized algorithms (ML models) and standardized testing to determine which method performs best across various scenarios. Our centralized, cloud-based platform for evaluating SOC estimation algorithms will accelerate research and development in battery management systems, benefiting researchers, engineers, and hobbyists. The project aims to deliver a public-facing platform that allows users to upload and benchmark SOC estimation algorithms through a scalable, parallelized cloud backend. The system will include user accounts, leaderboards, model visualization, and comparison.

3. Component Diagrams

Diagrams were taking up too much space in our core document, which violated the page limit. Please refer to the following figures available in the **Appendix**, located [here](#).

Figure 3.1 - Component Diagram

Figure 3.2 - Code Structure of Battery SOC Evaluation Algorithm (Python Version)

Single high-res image files available upon request. Please zoom in to read fine print.

4. Component Behaviours

Below, the components of the project are outlined by their: Intended Behaviours, Potential Undesired Behaviours, Implementations, APIs, Relationships to Requirements (which is normally its own section, but we received permission to format it thusly)

4.1 Frontend

Login Screen	
Normal Behaviour	Initially, unauthorized users will be directed to sign up. They will be unable to access the main features of the website until they do so. The login/registration screen captures inputted user credentials, sanitizes them, and sends an HTTPS request to the Application Server to create a new user account or authenticate an existing one. When a user session expires due to a predefined period of inactivity or manual logout, the user is then returned to the login screen to reauthenticate on next visit.
Undesired Behaviour	• The user must not be able to directly call links and access unauthorized pages, these attempts will direct the user to the login screen. • Improper input sanitization could invite injection attacks • During signup, no checking of existing accounts with inputted email or username could lead to a partial record or database error
Implementation	Either have user create an account and call POST command to store this account in the backend or have user login and call POST command to verify the credentials. Store login as an HTTP cookie and set <code>isLoggedIn</code> useState to true. Check the login on load time with a <code>useEffect</code> hook so user does not have to sign in on page refreshes

API	/users/login (POST) - The inputted email or username and password will be stored in the request body. Either a successful status code in the response body signals the app to navigate to the home page or an error will display on the user's screen, and will remain on the login page - The response stores a session cookie in the user's browser, which it uses for subsequent requests. The successful status code in the response body signals the app to navigate to the home page, or again, an error will display on the user's screen, and will remain on the login page /users/register (POST) - The inputted user info such as first name, last name, email, username, and password will be stored in the request body. - The response stores a session cookie in the user's browser, which it uses for subsequent requests
Requirement Relationships	Hosting a User Platform (P0) : The Login page allows users to login/signup to the app, upload models under their account and modify their uploaded models. Security (P1) : The login page ensures only registered users may access the system, and user accounts are protected by secure authentication methods, protecting system integrity and user data.

Leaderboard Page	
Normal Behaviour	A page on the website will display a leaderboard where users will be able to see the top 250 <u>public</u> user submitted models. The ranking of the leaderboard is sorted based on a model's weighted error score, computed by the Battery SOC Evaluator component. It also displays other metrics gathered from testing for each model on the leaderboard, where users can sort the models based on these metrics.
Undesired Behaviour	- The leaderboard rankings are outdated; the leaderboard does not regularly read the database to stay up to date. - Users who choose to submit their models <u>privately</u> will not have their model and performance results displayed on the public leaderboard. - Models that are later changed to private must be removed from leaderboard
Implementation	This component will read from the leaderboard database and get the first 250 model submissions. It will then parse through and display the fetched ranking data onto the leaderboard webpage. This includes username, model name, weighted error, and additional test metrics. The component will re-fetch leaderboards upon refresh. Rankings will be given weights based on the weighted error score being the RMSE.
API	/data/fetchLeaderboardData (GET) : The component makes a query call to the leaderboard database to get the first 250 ranked models along with its associated data.
Requirement Relationships	Hosting a User Platform, with Submission and Leaderboards (P0) : Enables an open platform for users to intuitively submit their models and compare the

	evaluated performance to other users via leaderboards for each battery type and make.
--	---

Model Comparison Page	
Normal Behaviour	Users will be able to select any 2 models from either their submissions or from the leaderboards page. They are able to filter which ones they wish to select using the Authors filter and the Institution filter. A graph will then display a visual comparison on every single error metric that has been calculated. Users will be able to download this file to understand how their models compare to other models/previous models they've made.
Undesired Behaviour	- Disallowing comparison between two of the exact same models to avoid redundancy and confusion. - Disallowing models that do not have valid results for comparison.
Implementation	The data will be collected by a GET API call to retrieve all the current leaderboard standings. This runs asynchronously in the background once the user logs in to improve performance, and this updates on a hard refresh from the browser. A GET API call is also used to retrieve all user submissions. This will be updated anytime the user uploads a new file which is handled by the useEffect hook. This information will be displayed on the frontend using React and the Recharts library will be used to plot the graphs.
API	/data/fetchVisualizerData (GET): Return the top 250 rows of the leaderboard as a list or rows in .json format. /user/submissions (GET): returns all valid (non-error) user submissions in a .json format.
Requirement Relationships	Leaderboard Visualization (P1) - Users are able to compare their model's performance against other users and their previous models. User's will be given the most up to date standings and submission when their browser loads and upon page refresh.

File Upload Page	
Normal Behaviour	The file upload page will have an element where the user can either drag-and-drop a file or browse their computer to select one. There is then a client-side check to verify the file is of an accepted type. The user can input a title, description, and set the model to public or private. If all necessary fields contain input, and the form is submitted, then a request is made to the Application Server , and then the page shows the user that there is an upload in progress.
Undesired Behaviour	- Unenforced file size limitations could lead to massive resource bottlenecks due to processing a large file - Unthorough file type constraints could invite malicious uploads
Implementation	The Library React Dropzone will be used to help with the overall look of the file submissions and create a clickable form in which matlab/python file may be

	submitted. ModelType will be selected from a dropdown list as well as a private/public privacy setting. An asynchronous handler will be in place to extract the university, and student name from the user as well as the modelType, privacy and modelName. File is checked to be a valid file creating the POST call for the model upload API call.
API	/model/upload (POST) The file is attached to the request body (with header 'Content-Type: multipart/form-data') with title, description, and status. User is collected from the context; response body contains the uploaded file's UUID.
Requirement Relationships	Model Submission File Reading System (P0) : The model upload page allows users to upload model files from their computer to be evaluated. Naming and setting visibility of their algorithms (P1) : The model upload page allows users to provide a title, description, and visibility status to their model while uploading. File Submission Interface (Drag-and-Drop Functionality) : The model upload page allows dragging and dropping files for upload and review/remove files before submission, whilst providing visual feedback (progress indicator, success/failure).

4.2 Backend

Application Server	
Normal Behaviour	The Application server is the main entry point for all client requests. It routes all HTTPS requests to appropriate modules and communicates with the Scheduler to assign tasks to clusters. The Middleware layer bridges between clients and business logic. It intercepts incoming requests to protected routes, extracting the token from the Authorization header, and validates it, rejecting requests with invalid tokens (error 401 Unauthorized). Upon successful authentication, an identity context is attached to the request, including the user's roles and permissions. There is then a check whether the user has sufficient roles/permissions to perform the request, rejecting if not (error 403 Forbidden). Upon successful authorization, the request payload is validated against a schema, rejecting if it fails (error 400 Bad Request). After passing validation, the middleware forwards the request to the relevant module (User management, File upload, etc). The middleware also logs events such as authentication and authorization successes and failures, model uploads and modifications, and errors or unexpected states. For file uploads, the Application server generates the user_info.json file using user data and uploaded metadata, compresses the script files with the json in a zip file, stores the zipped model in file storage, saves its file path and metadata in the database, and assigns the model to the scheduler for evaluation.
Undesired Behaviour	Without thorough security measures, unvalidated input or malicious file uploads could invite injection attacks

	• Race conditions with multiple accesses to shared resources at once • Memory bottleneck due to many users uploading files at once • On file upload, failing database write could lead to orphaned files
Implementation	The Application Server will be an Express.js app, while the database will be PostgreSQL. Authentication will be handled by the BetterAuth library.
API	The Application server routes all HTTP requests using different domains: • /users/* contains the functions related to user management, such as login and register. • /model/* contains the functions related to models, such as upload, retrieve, modify, and delete for uploading and retrieving models, as well as modifying their metadata and deleting them • /data/* contains the functions related to retrieving data, such as fetchLeaderboardData for fetching the data for the leaderboard, and fetchVisualizerData for fetching more specific/advanced data about a specific model, to be visualized.
Requirement Relationships	Security (P1) : The middleware ensures authorization and authentication of users using role-based mechanisms, and validates requests, protecting the system integrity, user data, and preserving resources. The Auth library ensures industry-standard security for accounts and protected endpoints. Hosting a User Platform (P0) : The application server handles all facets of app function, such as user management, file submission, and data queries.

Computing Clusters	
Normal Behaviour	Components sourced from multiple potential services (still in discussion) but may contain GPUs and CPUs from private computing resources, lab equipment, AWS, Google, and Canada Compute (now Digital Research Alliance). Listens to Scheduler and spins up container instances on demand. The primary job is to do evaluate battery algorithms (anticipating Neural Nets) quickly as a distributed network. Runs the Battery SOC Evaluation Algorithm and communicates in <i>Dask Dataframes</i> . Clusters are capable of being dynamically scaled, but we limit this behavior to the Scheduler's discretion.
Undesired Behaviour	• The distributed computing resources do not properly initiate container instances, or otherwise disregard Scheduler instructions • The distributed computing resources bottleneck and do not readily communicate with the Scheduler for load balancing
Implementation	Though we host on different platforms, we unify our interface for accessing Clusters through https://coiled.io/ , a paid service managing threads of distributed resources, also conveniently handling several Scheduler tasks.
API	Primary integration through Dask using PyTorch. See API at https://docs.coiled.io/examples/dataframes.html for integration and https://docs.coiled.io/user_guide/api.html for cluster management. Primary data being passed are Dask Dataframes.
Requirement Relationships	Parallelization (P0) : The Computing Cluster component is our distributed cloud network to run large or complex SOC algorithms efficiently.

Battery SOC Evaluation Algorithm (Python Version)	
Normal Behaviour	The original Battery SOC Evaluation Algorithm was rewritten from MATLAB to Python in order to take advantage of Python's faster processing time and parallelization capabilities through Dask. In a functional view, the Python version of this algorithm is expected to behave identically to the original MATLAB version, returning the same output given identical inputs. <i>Note: we treat the SOC algorithm and distributed system architecture as separate components, but the algorithm is modified to be parallelized using Dask Dataframes, so it is part of the parallelized system architecture in technicality. We give it component status to outline its importance and critical role.</i> • Evaluator takes in model submissions: The evaluator continuously waits to be fed a user submitted SOC estimation model. The evaluator is also given and holds onto the associated user's username and email. • Evaluates submitted models: The evaluator measures the submitted model's performance by running it against a series of test cases. • Update leaderboard: The evaluator records the submitted model's performance metrics along with the associated username and adds them to a record of existing model submission results. This change updates the leaderboard rankings. • Email results: The user that submitted a model will receive an email from the evaluator of their model's ranking (on the leaderboard) and performance after its evaluation.
Undesired Behaviour	• This Python version of the evaluator does not return identical performance results compared to the original MATLAB version. This would impact the reliability of its results and the accuracy of the leaderboard rankings. • The evaluator is unable to read model submissions.
Implementation	The following summarizes the classes used to run the Battery SOC Evaluation Algorithm in Python. Please see Figure __ for a class diagram of this component. • standardized_evaluation_tool.py: This file is the main script that orchestrates the evaluator. When live, the script handles all incoming submissions as assigned by the Scheduler. It also configures all variables, files, and dataframes needed for testing. For each queued submission, it passes the model data and user information to process_submission.py. • process_submission.py: This file is responsible for processing all the orchestrations that happen after a model is picked up. A confirmation email is sent to the user when the program has picked up their submitted model. Then it calls validate_submission.py, obtain_output_data.py, and create_figures.py for the evaluation. Once it's heard back from these functions, the program calculates some generalizing metrics that summarize the test results, such as average RMSE and the final score. It then saves the model evaluation findings to the database. This will then affect the leaderboard ranking results. Finally, the program sends an email to the user regarding their model's ranking, metrics, and performance results.

	• validate_submission.py: This file validates if the user submitted model is useable by the algorithm before testing. It does so by checking its file type(s) and testing if it can run without error within a certain time limit. • obtain_output_data.py: This file evaluates the user submitted model against battery SOC test cases and collects the resulting metrics. • create_figures.py: This file generates all the graphs of the resulting metrics that will be emailed to the user.
API	Input: This component takes in a zip file from the Scheduler that contains a user's submitted model and relevant user information. The model file(s) can be of MATLAB (.m) or Python (.py) format, while the user information is passed using a JSON file. /database/leaderboards (POST) Output: After this component is finished evaluating, it passes the model results along with the relevant user information to the database using a JSON file.
Requirement Relationships	Model Interpretation (P0) : Interpretation was the old goal, but the aim remains to translate existing MATLAB capabilities directly to Python, preserving code behavior and performance.

Parallelized System Architecture (Scheduler)	
Normal Behaviour	This component supports parallelization, enables distributed computing for load balancing, and arranging the order requests that are handled for optimal speed and user satisfaction (submission queue). Communicates directly with Computing Clusters & Application Server and is partially built-in to the Battery SOC Evaluation Algorithm (Python) with Dask Dataframes used as the <i>unified parallelized data structure</i>, replacing NumPy arrays, Python lists, and any parallelized matrix representations. • Listens to main Application Server: receives requests with associated data and assigns tasks to Computing Cluster “<i>optimally</i>” • Determines whether to use local computing (no internet access is the primary factor but also includes assessment of the busyness of the available computing clusters) to handle submission. • Splits up data to be evaluated in parallel, <i>handled automatically through Dask Dataframes paired with Coiled management. We have rewritten the Python code to allow for simple parallelization through Dask.</i> • Queues tasks and assigns them to the correct cluster threads or local threads. <i>Thread assignment logic handled by Coiled, but the queue and assignment is custom built.</i> • Offers load-balancing to guarantee throughput and user satisfaction. Communicates to Application Server information about the availability of compute power and monthly budget. This includes choices regarding: ▪ When to start/stop a cluster ▪ What CPU/GPU options to use ▪ How many computing resources to use in parallel for the current task in the queue, given final budget

	▪ When to discard tasks • Selects the evaluation algorithm, whether to be done on Python or MATLAB, if not specified by user. Parallelization is only possible in Python.
Undesired Behaviour	All the bullet points listed above must be met and must not incur egregious resource costs. A separation of software sourced from libraries or third parties is highlighted in small blue text. Violation of any of the normal behaviors is highly undesired, particularly, the following two scenarios are least ideal: • Partitions the workload unevenly, such that the most powerful processors are handling simpler tasks, creating bottlenecks • Leaves an active cluster running without closing it when feasible, incurring high allocation cost from service provider Note: problems arising from <i>parallelizing the data</i> falls under the purview of Dask + Coiled and is not a major concern, as is the nuances of scheduling threads on the cluster level, or the uptime of compute resources. These are provided to us as part of the service guarantees of VM providers and Coiled.
Implementation	In summary, parallelization of data will be handled through Dask. Management of the parallelized clusters, including communication and thread-scheduling will be handled through Coiled.io Management of the queue (separate from Coiled) will be implemented primarily in Scheduler.py, but note that this is currently a non-existent file as we have not yet separated the scheduler components from the App Server.
API	/scheduler/local is the intended endpoint after separation of the scheduler and application server, used for submitting local requests /scheduler/cluster is the intended endpoint after separation of the scheduler and application server, used for submitting local requests; requests will be redirected to local if it is determined that parallelization is infeasible. Primary integration through Dask using PyTorch. See API at https://docs.coiled.io/examples/dataframes.html for integration and https://docs.coiled.io/user_guide/api.html for cluster management. Primary data being passed are Dask Dataframes.
Requirement Relationships	Parallelization (P0): The Scheduler component satisfies all the tricky features of an ideal parallelized system: scheduling, queue, load-balancing for throughput, and language/algorithm selection as appropriate.

5. User Interface – Example with a User Story

Abby is in her second year of her master’s program and wants to be able to check the accuracy of the new AI model she has created by checking them against industry standards. On loading our app, Abby is greeted by a professional, confidence-inspiring hero page (**Figure 6.1**). The hero image is a faint abstract line graphic resembling the curves of battery charge over time. This floats in the background at low opacity, reinforcing the domain without overwhelming the interface.

An unauthorized user would then be prompted to click on a register/signup button as the

main features will not be shown to that user. The registration page (**Figure 6.2**) is a friendly environment that is emphasized by the contrast of the heropage; a nice colourful image of a desk on top of a bright background as opposed to the monotone colours from the hero image. There are friendly icons for each of the form inputs to reemphasize the joyful tone of the page.

Then there's the submission interface (**Figure 6.3**) which reinforces that sense of structure and control. The layout is not cluttered; content is centered and spaced intentionally. The drag-and-drop upload area visually communicates its function without explanation with its size and positioning to make it clear that uploading results is the primary action on the page. The presence of visible boundaries (cards, sections, or panels) gives Abby confidence that her data is being handled deliberately. Rather than feeling like she is "sending files into the void".

This portion supports both the submission and leaderboard pages (**Figure 6.4, 6.5**), the UI shifts from action to interpretation. The structured table layout mirrors the way results are typically presented in academic papers with rows, columns, aligned numbers making the information immediately familiar. The restrained use of color in the leaderboard further supports this narrative. Instead of loud highlights or gamified effects, subtle emphasis is used to guide attention. This makes the comparison feel analytical rather than competitive. She is also given a bunch of different options for filtering her results, such that she will not be overwhelmed by the number of rows available and can intuitively pinpoint her results to her own needs.

On the model comparison page (**Figure 6.6**), the UI shifts from presenting raw results to supporting interpretation, and this transition is handled with the same restraint and clarity as the rest of the platform. The side-by-side alignment of models immediately communicates that the purpose of the page is comparison, not exploration. By presenting multiple models within a single, structured view, the interface tells a clear story: performance is relative, measurable, and contextual, helping Abby understand not only how her model performs, but how and why it differs from industry standards.

Across all pages, the consistent typography and spacing act as a silent narrator. Headers clearly separate stages of the process, while body text remains readable and neutral. This consistency helps Abby move through the experience without reorienting herself at every step. The interface does not try to impress her with excess features. Instead, it earns her trust by staying clear, professional, and aligned with the way she already thinks as a researcher.

6. V&V Component Test plan

Battery SOC Evaluation Algorithm (Python Version)
--

Unit Tests	Since this component is a direct translation of the original MATLAB battery SOC evaluation algorithm, the Python evaluator will be tested primarily against the MATLAB implementation. The <i>unit test</i> framework is proposed for this. • Identical Functionality: To ensure the Python evaluator produces identical results to the MATLAB evaluator, we will implement multiple unit tests using several model submission cases (sourced from supervisor). For each case, we will run both evaluators independently and compare the computed metrics and resulting rankings. The Python version will be considered correct if it matches the MATLAB results. Specifically, we will look for differences between the resulting error metrics, weighted scoring, and complexity scoring. We will also test edge cases using advanced model submissions provided by our supervisor. • Ranking Output: Continuing the idea of guaranteeing identical results, we will feed both evaluators a series of models and check if the Python produces the same model rankings (a cumulative result from the prior metrics) as MATLAB. To check for edge cases, we will employ a test case that has a set of <i>near identical models</i> to tease out any variation • Data Handling: Because the implementations use different languages, we will verify that data loaded in Python is handled identically to MATLAB. Unit tests will confirm that file loading produces equivalent data structures, including indexing, numeric representation, and NaN handling. • Full Integration: Verify that this component is fully integrated and functioning correctly within the overall system. This will be tested by sending model submissions from the Scheduler to the evaluator and confirming that the evaluator can correctly load, process, and record each submission's results. Edge cases such as empty files, invalid submission formats, and submissions that exceed computation time limits will also be tested to ensure the system handles these scenarios appropriately.
Performance Tests and Metrics	Identical Functionality: The outputs (errors and accuracy) must match within floating-point precision limits, as even tiny differences in RMSE calculations could change leaderboard rankings. These are the following numerical accuracy metrics we will use (for checking error metric calculations, weighted scoring): • Maximum Absolute Error: $\max(Python_Output - MATLAB_Output)$ For measuring the largest raw difference between any Python and MATLAB output. Any differences larger than 1e-4 would be of concern due to the sensitivity of the RMSE values. • Maximum Relative Error: $\max(Python_Output - MATLAB_Output / MATLAB_Output)$ For metrics that are very sensitive and usually result in values very close to zero. This measurement helps us detect differences more proportional to their true values. Any differences larger than 1e-4 would be of concern. • Correlation Coefficient: For measuring the difference between multiple metric outputs (i.e. predicted SOC curve), this can help us see how well the shape/pattern of outputs match. Any correlation coefficient that is smaller than 0.9999 would be of concern.

	• Ranking Models: The Python evaluator must produce identical rankings to the MATLAB version across multiple model submissions. Since any differences would directly impact the public leaderboard, no differences are acceptable. Runtime: It is acceptable for the Python evaluator to run at approximately the same speed as the MATLAB version, whether slightly faster or slower. This is because the Parallelized System Architecture component ensures that this component will run faster than the older MATLAB version. We have typically seen much better speed and utilization with Python. • Metric of Total Evaluation Time and CPU/Memory Utilization															
Test Results	Identical Functionality and Data Handling: The metric results between the MATLAB evaluator and the converted Python evaluator were very different (especially the RMSE metrics i.e. Weighted Error). The difference in metrics between the two evaluations are larger than 1e-4. We suspect that this failed unit test case has to do with error with the floating-point numbers in the test data, as well as the nature of random seed generation differences between MATLAB and Python. These caveats are further explained in the “SRS and Project Plan” document. Note: Ran out of time to fix this issue due to losing a group member. Ranking Output: We fed both the Python evaluator and MATLAB evaluator two different SOC estimator models (Python model versions for Python evaluator and vice versa) and checked if the Python evaluator ranked the models in the same order as the MATLAB evaluator. Form our unit test, we saw that the MATLAB evaluator ranked the model “Extended Kalman Filter” as better than the model “Coulomb Counter”. In comparison, the Python evaluator also ranked these two models in the same order (both the Python and MATLAB versions of these models were provided by Dr. Kollmeyer). MATLAB ranking: <table><tr><th>Submission</th><th>Author</th><th>Affiliation</th><th>Model_Name</th><th>Weighted_T</th></tr><tr><td>#####</td><td>Ahnaf Rahi</td><td>McMaster</td><td>Extended Kalman Filter</td><td>10.946</td></tr><tr><td>#####</td><td>Ahnaf Rahi</td><td>McMaster</td><td>Coulomb Counter</td><td>12.631</td></tr></table> Python Ranking:	Submission	Author	Affiliation	Model_Name	Weighted_T	#####	Ahnaf Rahi	McMaster	Extended Kalman Filter	10.946	#####	Ahnaf Rahi	McMaster	Coulomb Counter	12.631
Submission	Author	Affiliation	Model_Name	Weighted_T												
#####	Ahnaf Rahi	McMaster	Extended Kalman Filter	10.946												
#####	Ahnaf Rahi	McMaster	Coulomb Counter	12.631												

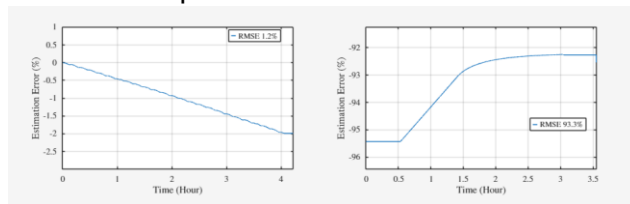
Submission	Author	Institution	Model Name	Model Type	Visibility	Submitted at	Completed at	Weighted Error ▼
13	Test1 Test	Test	Coulomb Counter Baseline	Coulomb Counter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	4.8
14	Test1 Test	Test	Coulomb Counter with OCV Correction	Coulomb Counter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	3.95
18	Test1 Test	Test	FNN Quick Estimator	FNN	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	3.1
2	Test1 Test	Test	EKF-UKF Dual Estimator	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	2.88

Note: This table shows rank in reverse, better models have a smaller weighted error. Extended Kalman Filter having a smaller Weighted Error than Coulomb Counter means that Extended Kalman Filter is ranked higher.

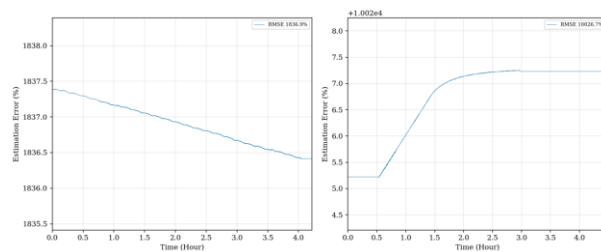
Graph Shape Comparison:

We checked the graph results between the MATLAB evaluator and the converted Python evaluator to see if they exhibited similar trends using the same model. We found that the trends resulting from the Python evaluation closely resembled the trend shape of the MATLAB evaluation. The following are examples of these graph trend comparisons.

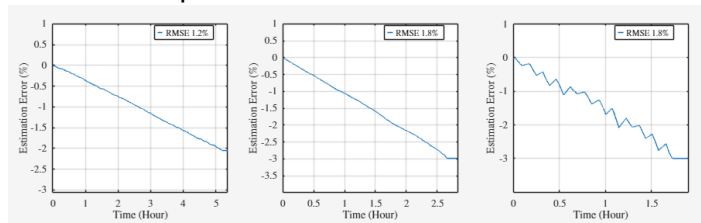
MATLAB Graph 1:



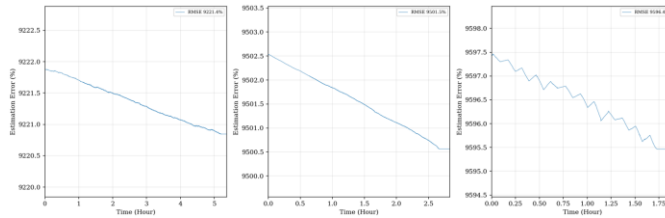
Python Graph 1:



MATLAB Graph 2:



Python Graph 2:



Full Integration:

Confirmed that the Python evaluator is fully integrated into our system architecture and can be called from our backend with no problem. The evaluator can load, process, and record each submission's results without running into any errors. We tested edge cases such as empty files, invalid submission formats, and submissions that exceed computation time limits, which all passed and were accounted for in our code.

Parallelized System Architecture (Scheduler) + Computing Clusters

Unit Tests

Unit tests are best for testing one isolated component at a time, yet the **Scheduler** communicates with **Clusters** in distributed fashion. Network timing, partial failures, clock skew, retries, etc. are inherently nondeterministic. It is therefore difficult to gather consistent results. *Deference to performance tests.*

It is possible to do unit testing solely on the Scheduler (non-Coiled implementation), wherein the queue and resource management logic is the primary focus. While we defer heavily to performance tests, we **may** implement the following flavor of unit tests (up to discretion):

- **Scheduling logic:** given a snapshot of cluster availability, costs, and a job request, the scheduler selects the most cost-effective clusters, or instantiates new clusters, or rejects the request within our final budget
- **Malleability:** transition tasks from cluster to local processing as intended
- **Retry behavior:** tasks transition correctly through lifecycle states (Pending → Scheduled → Running → Failed), and retry requests are levied correctly

Note: as we are using coiled, the **scheduling logic** and part of the **retry behavior (for different containers)** is implemented through them. *We were under the impression that coiled was a less-capable service, so we only expected to offshore about 30% of the scheduler's tasks to it. However, it turns out we ended up relying on coiled for more than 70% of the scheduler functionality.*

Therefore, for these tests, we simply demonstrate that we have configured coiled to behave exactly as specified by showing runs on multiple models and directly comparing the results therein.

For feedback, you **suggested that we conduct unit tests regardless of nondeterministic behavior**, so we have come up with new tests adapted to our new

	P0-P2 parallelization levels. We borrow some ideas used in our performance and metrics testing, and thereby also present parallelization benefits and cost. Note: Malleability can be measured by how our scheduler chooses local or parallel processing, as determined by our complexity grader P0-P2 parallelization levels correspond to native MATLAB post-evaluation scores from 1-9, shown below, demonstrating that our scheduler dynamically chooses correctly based on model complexity via heuristics.
Performance Tests and Metrics	Metrics: latency (time), load-balancing (percentile), cost-effectiveness (throughput per cost), and parallelization benefit (time) Proposal to use JMeter Distributed Testing Suite (link) to test latency as a measure of time to communicate between clusters, and load-balancing as a percentile of resource consumption. Stress test will involve Scheduler (controller node) sending clusters (worker nodes) as a substantial SOC model to evaluate, provided by supervisor. We will use Latency, and Load Time expressed as a percentile of total elapsed time. We will compare total elapsed time to the elapsed time of a single core (local) processor initiated on an isolated node to determine parallelization benefit. Note: we've already performed tests (see class demo video) to verify that a parallelization benefit of 4x better time performance compared to single core MATLAB is achievable. Cost effectiveness to be determined by final budget compared to average throughput in JMeter [(number of requests) / (total time) / total cost]. Likely using JMeter with Python runtime via (https://github.com/eldaduzman/pymeter). Note: we swapped to PyTest, Dason is the one running these tests and he is more familiar with PyTest
Test Results	All results, in detail, are included in the Jupyter Notebook in "Parallelization Workbench" of our Git Repo, available here. Included are explanations of what models we used, and each of our parallelization levels P0-P2. UNIT TESTS Individually Conducted Unit Tests on several combination of our 4 sample models run locally with P0, in parallel with P1 (2 workers, and our most complex model (Kalmann) run with P2 (up to 6 workers): As shown, local run performs without issue:


```
# LOCAL TESTS
%run standardized_evaluation_tool.py "user1" "P0"
```

```
Loading test data...
Starting the blind modeling tool...
Zip extracted successfully.
Model file found!
Starting validation...
Starting validation process...
Validation passed!
Running test cycles...
Generating figures...
Calculating scores...
Final weighted score: 5669.9283
{"error": false, "results_path": "w:\\Academia\\McMaster University\\
Results written to: user1\\results\\results.zip
```

Coiled performs admirably, with worker scheduling done automatically on the best available ones for this region. This is deliberation process is internally done and not logged:

```
%run standardized_evaluation_tool.py "user1" "P1"
```

✓ 4m 53.3s

```
Loading test data...
Starting the blind modeling tool...
Zip extracted successfully.
Model file found!
Starting validation...
Starting validation process...
Validation passed!
Running test cycles...
Scheduler Activated - Processing mode P1
Spinning up a cluster...
[2026-04-07 15:05:02,112][INFO    ][coiled] Fetching latest package priorities...
[2026-04-07 15:05:02,124][INFO    ][coiled.package_sync] Resolving your local .venv Pyt
[2026-04-07 15:05:02,292][INFO    ][coiled.package_sync] Scanning 104 python packages..
[2026-04-07 15:05:02,680][INFO    ][coiled] Running pip check...
[2026-04-07 15:05:03,796][INFO    ][coiled] Validating environment...
[2026-04-07 15:05:06,959][INFO    ][coiled] Creating wheel for W:\\Academia\\McMaster Uni
[2026-04-07 15:05:07,418][INFO    ][coiled] Creating wheel for W:\\Academia\\McMaster Uni
[2026-04-07 15:05:07,703][INFO    ][coiled] Uploading coiled_local_SETool_Python...
[2026-04-07 15:05:08,907][INFO    ][coiled] Uploading coiled_local_processing...
[2026-04-07 15:05:09,650][INFO    ][coiled] Requesting package sync build...
[2026-04-07 15:05:10,182][INFO    ][coiled] Creating Cluster (name: soc-data-cluster, h
Scattering data to workers...
Force-syncing Model.py to worker disks...
Submitting task...
Task Parallelization Complete, check for errors. Cleaning up...
[2026-04-07 15:09:17,230][INFO    ][coiled] Cluster 1550213 deleted successfully.
Generating figures...
```

We tested 4 different models (compared 3) and got the following insights:

User 1 with Basic CCounter

P1 / P0 = 0.034 in terms of avg. CPU utilization (%)

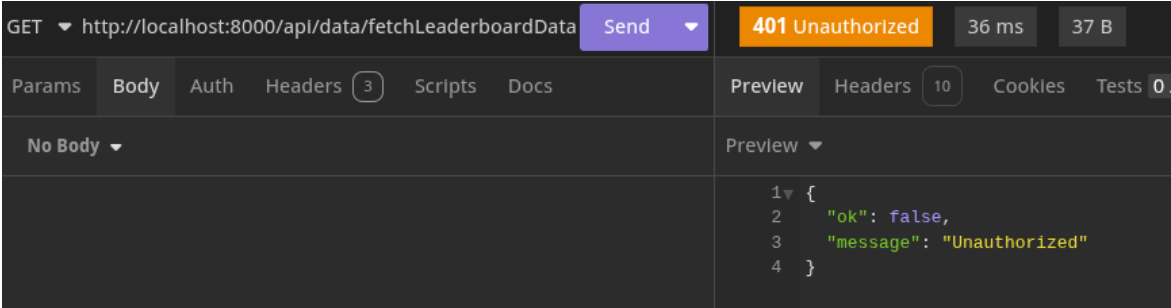
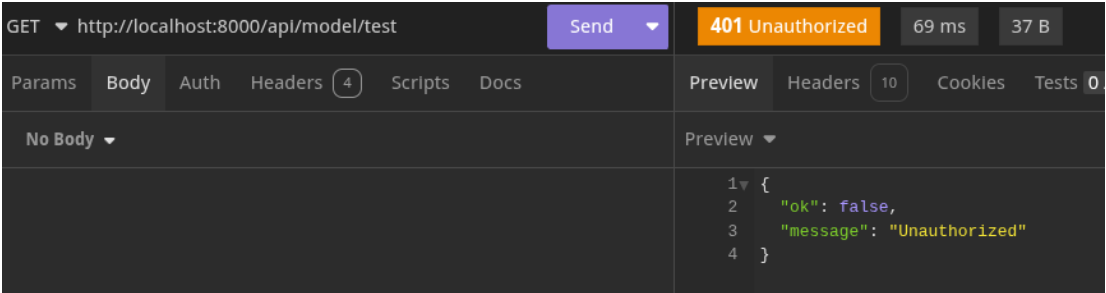
P1 / P0 = 1.054 in terms of avg. memory utilization (%)

P1 / P0 = 2.159 in terms of elapsed time (s)

P1 / P0 = 0.073 in terms of est. CPU usage time (s)

P1 / P0 = 2.277 in terms of est. memory usage time (s)

	The cost to run P1 was 0.01 CAD User 3 with Stochastic CCounter P1 / P0 = 0.01 in terms of avg. CPU utilization (%) P1 / P0 = 1.138 in terms of avg. memory utilization (%) P1 / P0 = 1.444 in terms of elapsed time (s) P1 / P0 = 0.015 in terms of est. CPU usage time (s) P1 / P0 = 1.643 in terms of est. memory usage time (s) The cost to run P1 was 0.05 CAD User 4 with Kalmann P1 / P0 = 0.008 in terms of avg. CPU utilization (%) P1 / P0 = 1.446 in terms of avg. memory utilization (%) P1 / P0 = 1.337 in terms of elapsed time (s) P1 / P0 = 0.011 in terms of est. CPU usage time (s) P1 / P0 = 1.933 in terms of est. memory usage time (s) The cost to run P2 was 0.16 CAD Findings: We verify that typically parallelization is bottle-necked by memory. We are able to trade an average of 1.213 times memory utilization compared to a local model, in exchange for roughly 57.692 times free CPU utilization Elapsed time of both CPU and memory follow a similar trend. Elapsed time ratio is shown to decrease towards 1.0 for more complicated models. It costs roughly 0.073, and prolongs the execution of a model by 1.647 Stress Tests showed inconsistent results (as expected of concurrent setup), but we consistently saw that P0-level models were best served on local processing, especially if the submissions are sporadic, as the cost to orchestrate even two workers is expensive. Finally, showed that we have malleability in our Complexity Grader, which was able to correctly predict $\frac{3}{4}$ of our Python models' complexities as scored by the evaluator after interpolating our 3 tiers to scores between 0-10. The only model to be correctly graded had a complication with MATLAB data files which we admit is very challenging to resolve with a heuristics-based approach, as all MATLAB code essentially becomes a black box.
Application Server	
Unit Tests	We will test various responsibilities of the Application Server to verify behaviour:

	• Authentication: We will verify that missing, expired, or malformed tokens return error codes, while the request proceeds with a valid token. • Authorization: We will confirm users without correct permissions cannot access protected endpoints; admin routes reject non-admin users. We also check ownership of models, to verify only model owners can modify their details. • Validation: Unit tests will ensure that malformed requests are rejected before proceeding. We will verify that required fields are present, structure is correct, formats of certain fields (for example, email) are as expected. • Logging: Unit tests will verify that certain actions are logged, such as failed login attempts, authorization failures, and model evaluations.
Performance Tests and Metrics	Performance will be measured using mean request latency for each request, and request throughput (successful requests per second) under simulated use conditions. Likely using JMeter as in the Scheduler .
Test Results	• Authentication — Attempt to access site pages (besides Home and Help) while unauthenticated ○ Expected Outcome: Redirects to login page ○ Actual Outcome: Matches expected • Authentication — Accessing protected endpoints (such as /api/model/delete) while unauthenticated ○ Expected Outcome: Returns error 401 Unauthorized, user cannot access endpoint ○ Actual Outcome: Matches expected  • Authentication — Malformed, invalid, or expired session token passed to application server in request ○ Expected Outcome: Returns error 401 ○ Actual Outcome: Matches expected  • Authorization— Banned user attempts to access protected endpoint ○ Expected Outcome: Returns error 403, blocks user from accessing endpoint

- **Actual Outcome:** Matches expected

GET http://localhost:8000/api/model/testBan Send 403 Forbidden 26 ms 34 B

Params Body Auth Headers 4 Scripts Docs Preview Headers 10 Cookies Tests 0

No Body ▾ Preview ▾

```
1 {  
2   "ok": false,  
3   "message": "Forbidden"  
4 }
```

- **Authorization— User attempts to call `/api/model/delete` endpoint to delete a model they don't own**
 - **Expected Outcome:** Model is not deleted, returns error 404 "Model not found"
 - **Actual Outcome:** Matches expected

DELETE http://localhost:8000/api/model/delete/4 Send 404 Not Found 59 ms 28 B

Params Body Auth Headers 4 Scripts Docs Preview Headers 10 Cookies Tests 0

No Body ▾ Preview ▾

```
1 {  
2   "error": "Model not found."  
3 }
```

- **Authorization— Non-admin user attempts to access role-protected endpoint with insufficient role**
 - **Expected Outcome:** Returns error 403 Forbidden, user cannot access endpoint
 - **Actual Outcome:** Matches expected

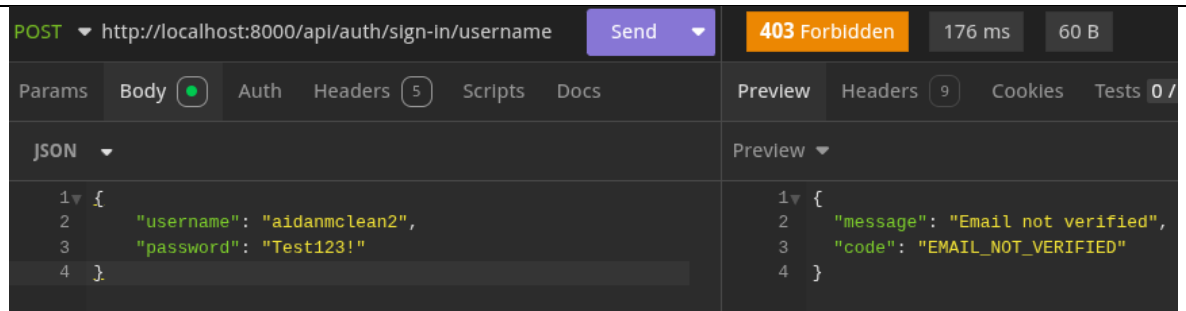
GET http://localhost:8000/api/model/test Send 403 Forbidden 20 ms 34 B

Params Body Auth Headers 4 Scripts Docs Preview Headers 10 Cookies Tests 0

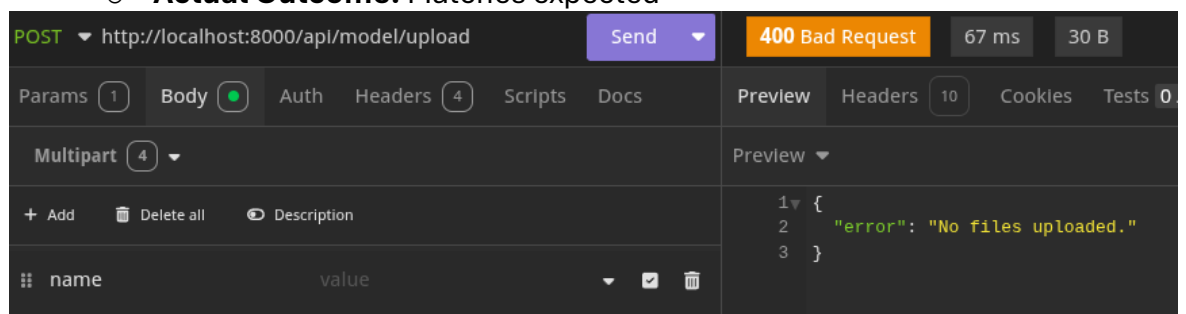
No Body ▾ Preview ▾

```
1 {  
2   "ok": false,  
3   "message": "Forbidden"  
4 }
```

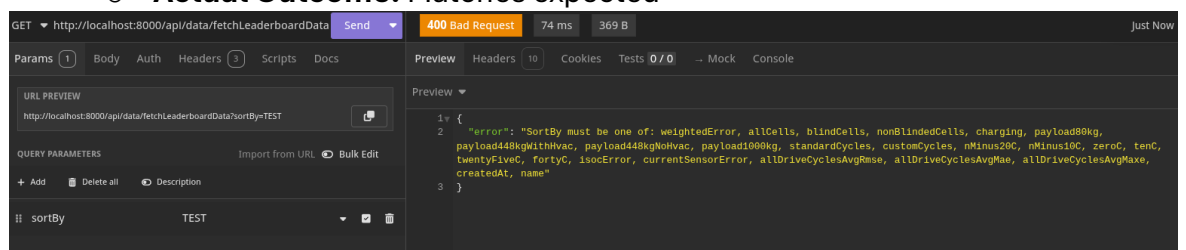
- **Authorization— User attempts to login without verifying email after signup**
 - **Expected Outcome:** User is blocked from signing in and a new verification email is sent
 - **Actual Outcome:** Matches expected



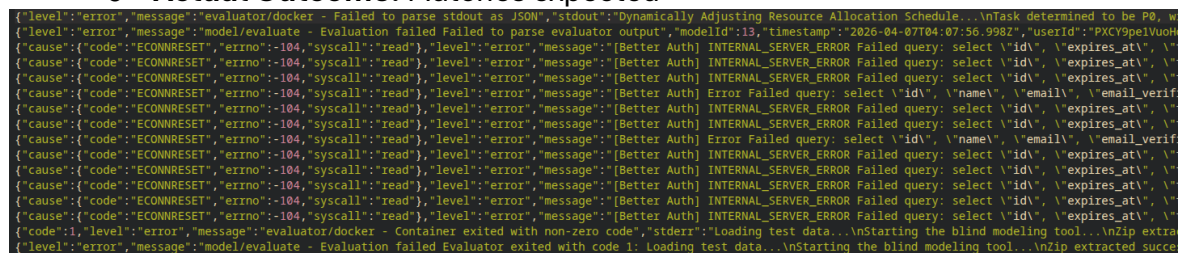
- **Validation— ModelType, name, description, or files missing from request body of *api/model/upload***
 - **Expected Outcome:** An error 400 is thrown and model is not uploaded or evaluated
 - **Actual Outcome:** Matches expected



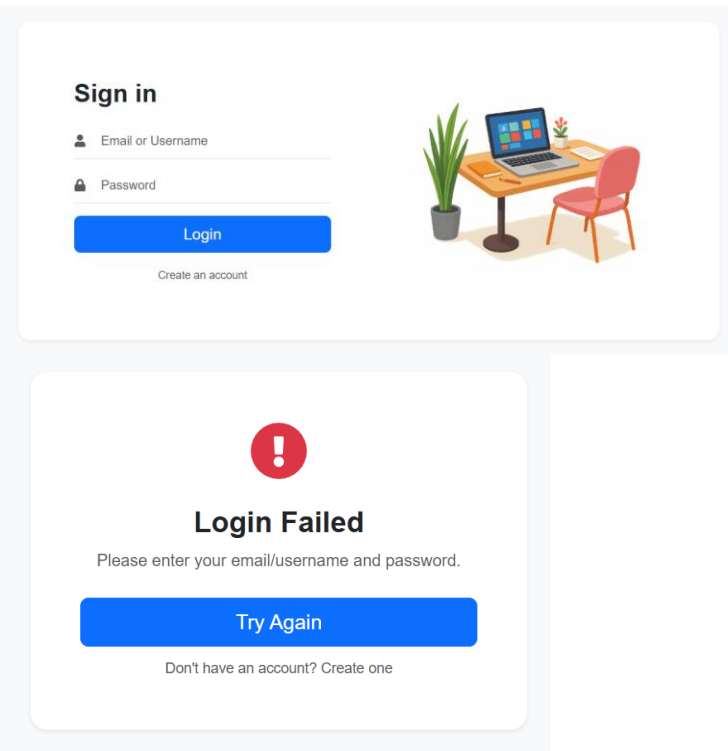
- **Validation— */api/data/fetchLeaderboardData* query parameters fall outside of expected values**
 - **Expected Outcome:** An error 400 is thrown and data is not returned
 - **Actual Outcome:** Matches expected



- **Logging— An error occurs on the server (something goes wrong)**
 - **Expected Outcome:** All logs with ‘Error’ level are appended to separate error logfile in addition to combined logfile
 - **Actual Outcome:** Matches expected



Login Page

Unit Tests	• Valid inputs: We validate that input fields only accept valid inputs (i.e. strings of valid length and formatted email addresses), verify that inputs are sanitized. The frontend first checks to see if they have submitted valid credentials before querying the backend. • Incorrect sign in details: any potentially viable username and password combinations will query the database. If not found in the database they will show an error message pertaining specifically towards the email/password or username/password combination entered. • Successful Login: Upon a successful login the page redirects you back to the leaderboards page with your full name present as the icon replacing the login and register buttons.
Performance Tests and Metrics	Performance will be based on minimizing database queries for the backend by allowing the frontend to deal with valid inputs. Performance will also be measured through accuracy of the error messaging allowing the user to understand where the issue resides in the credentials.
Test Results	Valid inputs: In the following examples, we show two attempts of logging in, one with no credentials involved, and one with a username under 8 characters and their respective error messages. 

Sign in

 asdf



Login

[Create an account](#)



Incorrect sign in details: The two examples given give the text fields inputted for a username and email login and their respective error message.

Sign in

 asdf@asdf.com



Login

[Create an account](#)



Login Failed

Invalid email or password

Try Again

[Don't have an account? Create one](#)

Sign in

 asdfasdf



Login

[Create an account](#)



Login Failed

Invalid username or password

Try Again

[Don't have an account? Create one](#)



Login Failed

Username is too short

Try Again

[Don't have an account? Create one](#)

Successful Login:

	SOCAIgoTestingPlatform Leaderboards Submit Model Submissions Model Comparison Help TTTest1 TestLogout Leaderboards Contact Administrator Filters: Filter by Author Filter by Academic Affiliation Model Type All Authors All Academic Affiliations All Model Types <table><tr><th>Submission</th><th>Author</th><th>Institution</th><th>Model Name</th><th>Model Type</th><th>Visibility</th><th>Submitted at</th><th>Completed at</th><th>Weighted Error</th><th>All Cells</th><th>Blind Cells</th><th>Non-Blinded Cells</th><th>Charging</th><th>80k Payk</th></tr><tr><td>1</td><td>Test1 Test</td><td>Test</td><td>EKF Battery SOC Estimator</td><td>Extended Kalman Filter</td><td>Public</td><td>4/5/2026, 5:38:57 PM</td><td>4/5/2026, 5:38:57 PM</td><td>2.34</td><td>2.1</td><td>2.89</td><td>1.95</td><td>1.8</td><td>2.1</td></tr><tr><td>2</td><td>Test1 Test</td><td>Test</td><td>EKF-UKF Dual Estimator</td><td>Extended Kalman Filter</td><td>Public</td><td>4/5/2026, 5:38:57 PM</td><td>4/5/2026, 5:38:57 PM</td><td>2.88</td><td>2.7</td><td>3.15</td><td>2.5</td><td>2.2</td><td>2.7</td></tr><tr><td>3</td><td>Test1 Test</td><td>Test</td><td>Adaptive EKF v3</td><td>Extended Kalman Filter</td><td>Public</td><td>4/5/2026, 5:38:57 PM</td><td>4/5/2026, 5:38:57 PM</td><td>1.95</td><td>1.8</td><td>2.2</td><td>1.68</td><td>1.55</td><td>1.8</td></tr><tr><td>4</td><td>Test1 Test</td><td>Test</td><td>LSTM SOC Predictor v2</td><td>LSTM</td><td>Public</td><td>4/5/2026, 5:38:57 PM</td><td>4/5/2026, 5:38:57 PM</td><td>1.87</td><td>1.7</td><td>2.1</td><td>1.6</td><td>1.4</td><td>1.7</td></tr></table>	Submission	Author	Institution	Model Name	Model Type	Visibility	Submitted at	Completed at	Weighted Error	All Cells	Blind Cells	Non-Blinded Cells	Charging	80k Payk	1	Test1 Test	Test	EKF Battery SOC Estimator	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	2.34	2.1	2.89	1.95	1.8	2.1	2	Test1 Test	Test	EKF-UKF Dual Estimator	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	2.88	2.7	3.15	2.5	2.2	2.7	3	Test1 Test	Test	Adaptive EKF v3	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	1.95	1.8	2.2	1.68	1.55	1.8	4	Test1 Test	Test	LSTM SOC Predictor v2	LSTM	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	1.87	1.7	2.1	1.6	1.4	1.7
Submission	Author	Institution	Model Name	Model Type	Visibility	Submitted at	Completed at	Weighted Error	All Cells	Blind Cells	Non-Blinded Cells	Charging	80k Payk																																																										
1	Test1 Test	Test	EKF Battery SOC Estimator	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	2.34	2.1	2.89	1.95	1.8	2.1																																																										
2	Test1 Test	Test	EKF-UKF Dual Estimator	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	2.88	2.7	3.15	2.5	2.2	2.7																																																										
3	Test1 Test	Test	Adaptive EKF v3	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	1.95	1.8	2.2	1.68	1.55	1.8																																																										
4	Test1 Test	Test	LSTM SOC Predictor v2	LSTM	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	1.87	1.7	2.1	1.6	1.4	1.7																																																										
File Upload Page																																																																							
Unit Tests	Required Fields: We ensure that all required fields have been filled out before files can be submitted Check for signin. User must be logged in before submitting a model, any attempts to access the submit model page unauthorized results in a redirect to the login page Valid file type: We validate that the file input field only accepts a finite, specified number of files of expected types (.mt or .py). We validate that the other metadata fields only accept valid inputs of a specific length, and that inputs are sanitized. Submission: We validate that submitting the file upload form sends the uploaded file(s) and metadata successfully in the request body and that the page updates according to request success or failure.																																																																						
Metrics	Performance will be measured by ensuring safety of the database by disallowing users to input models with invalid fields potentially corrupting the database. Performance will also be measured through limiting calls to the backend for incorrect file types that cannot be tested.																																																																						

Test Results

Required Fields:

Description is required.



Model Name *

asdf

Description *

Enter model description

Model Type

Kalman Filter



Evaluation Level

NONE



Private ☐

Please upload a model file.



Model Name *

asdf

Description *

asdf



Model Type

Kalman Filter



Evaluation Level

NONE



Private ☐

Submit Model

Check for signin. The following image is the resulting redirect that occurs when an unauthenticated user attempts to access the page.

Sign in

 Email or Username

 Password

Login

[Create an account](#)



Valid file type:

Upload failed: {"error": "Invalid file type. Only Python (.py) and MATLAB (.m) files are allowed."}

Model Name *

Paarth's PDF

Description *

a PDF file

Model Type

Machine Learning

Evaluation Level

dynamic

Private ☐

Submit Model

Capstone 2026 How to Best Present Yourself.pdf

Submission:

Submission	Model Name	Model Type	Status	Visibility	Submitted at	Completed at	Weighted Error	All Cells	Blind Cells	Non-Blinded Cells	Charging	80kg Payload	448kg Payload with HVAC
1	Aidan's O(n ²) CC Test Model	Coulomb Counter	ready	Public	4/7/2026, 3:07:59 AM	4/7/2026, 3:18:22 AM	30.769	35.152	20.254	40.118	10.369	42.792	20.254

Leaderboard Page

Unit Tests

Public Models only: ensure that all models selected for the leaderboards are public models. The following is the database query that is being used in order to select the public models and attach them to their respect user to show their name and academic affiliation. (this has been translated from drizzle-orm to SQL for simplicity sake)

```
SELECT u.id, u.name, u.first_name, u.last_name, u.academic_affiliation , m.* FROM "user" u INNER JOIN models m ON m.owner_user_id = u.id WHERE m."isPrivate" = f
```

Leaderboard fetching: ensure that the leaderboard is fetched once upon the page load. The following is a react component that is mounted on top of the page. The [] at the very end of the function means it is only called once and the fact that it is mounted mean it is only called when the app is opened.

```
useEffect(() => {
  const fetchData = async () => {
    try {
      setLoading(true);
      const response = await fetch("/api/data/fetchUserModelJoin");
```

	<pre>if (!response.ok) throw new Error(`Error: \${response.status}`); const data = await response.json(); console.log(data.data); const formatted = formatData(data.data); setOriginalData(formatted); setFormattedData(formatted); setUniqueUsernames(getUniqueUsernames(data.data)); setUniqueUniversities(getUniqueUniversities(data.data)); console.log(uniqueUsernames, uniqueUniversities) } catch (err) { setError(err.message); } finally { setLoading(false); } }; fetchData(); }, []);</pre>																				
Metrics	Improve page loading times by having the leaderboard be fetched once upon the page load as a background process																				
Results	The following result is a collection of 25 rows selected from a database from a total of 30 submissions, only the public models are displayed on leaderboards. <table><tr><th>id</th><th>first_name</th><th>last_name</th><th>academic_affiliation</th><th>isPrivate</th></tr><tr><td>25</td><td>Paarth</td><td>Kadakia</td><td>McMaster University</td><td>f</td></tr><tr><td>...</td><td></td><td></td><td></td><td></td></tr><tr><td>1</td><td>Aidan</td><td>Mclean</td><td>McMaster University</td><td>f</td></tr></table> (25 rows) Leaderboard is now able to be used without having to load anytime page is accessed.	id	first_name	last_name	academic_affiliation	isPrivate	25	Paarth	Kadakia	McMaster University	f	...					1	Aidan	Mclean	McMaster University	f
id	first_name	last_name	academic_affiliation	isPrivate																	
25	Paarth	Kadakia	McMaster University	f																	
...																					
1	Aidan	Mclean	McMaster University	f																	

Leaderboards

Contact Administrator

Filters:

Filter by Author

Filter by Academic Affiliation

Model Type

All Authors

All Academic Affiliations

All Model Types

Submission	Author	Institution	Model Name	Model Type	Visibility	Submitted at	Completed at	Weighted Error	All Cells	Blind Cells	Non-Blinded Cells	Charging	80kg Payload	448kg Payload with HVAC
1	Test1 Test	Test	EKF Battery SOC Estimator	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	2.34	2.1	2.89	1.95	1.8	2.1	2.65
2	Test1 Test	Test	EKF-UKF Dual Estimator	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	2.88	2.7	3.15	2.5	2.2	2.75	3.1
3	Test1 Test	Test	Adaptive EKF v3	Extended Kalman Filter	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	1.95	1.8	2.2	1.68	1.55	1.85	2.3
4	Test1 Test	Test	LSTM SOC Predictor v2	LSTM	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	1.87	1.7	2.1	1.6	1.4	1.75	2.2
5	Test1 Test	Test	Bidirectional LSTM SOC	LSTM	Public	4/5/2026, 5:38:57 PM	4/5/2026, 5:38:57 PM	1.72	1.58	1.92	1.48	1.3	1.65	2.05

Model Comparison Page

Units Tests

- **Comparison of unique valid models:** Models without valid or complete evaluation results are excluded from comparison, and the platform verifies that both selected models are unique.
- **Selection of two models:** Ensure two models are selected

Metrics

The metrics captured should be the meaningful and accurate display of two distinct models, preventing users from comparing a model against itself to avoid redundancy and confusion.

Comparison of unique valid models:

Cannot compare two of the same models

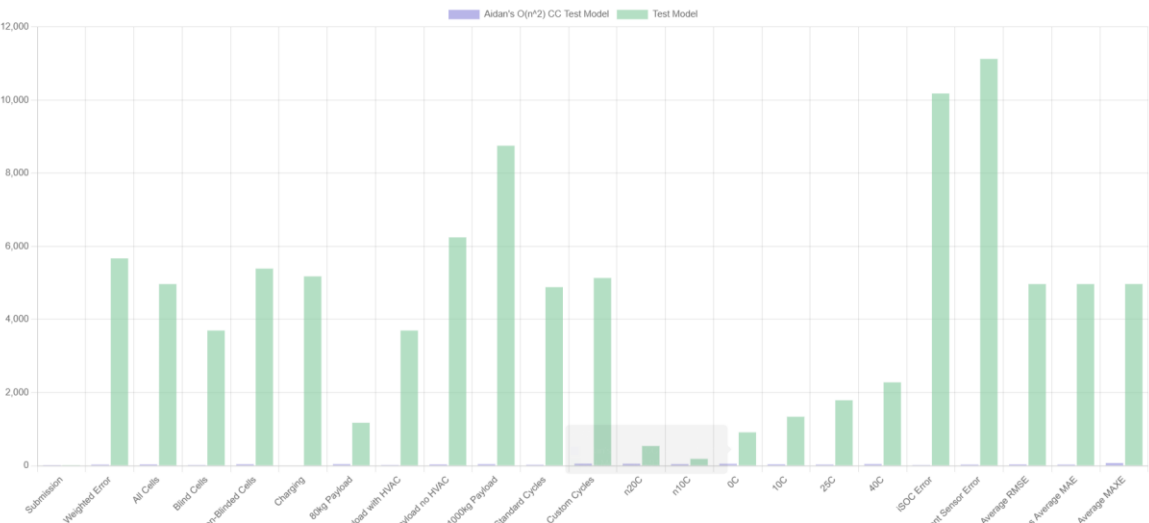


Selection of two models:

Must select more than one model type



Success:



7. Appendix

Figure 3.1 - Component Diagram

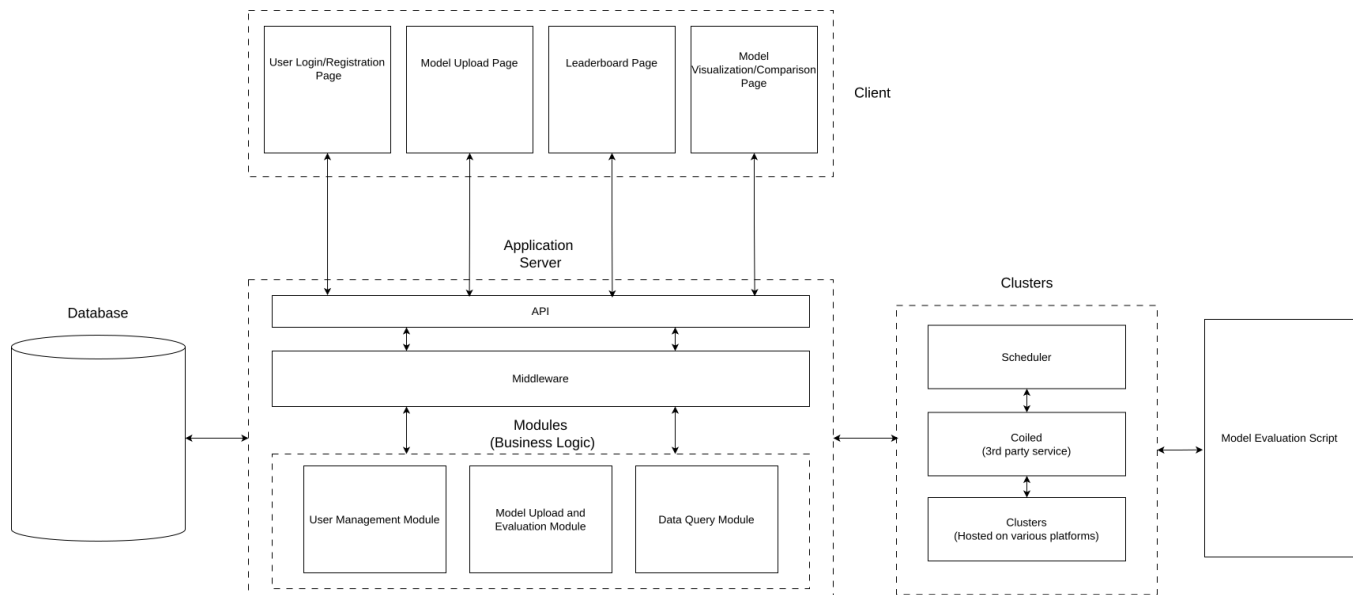


Figure 3.2 - Code Structure of Battery SOC Evaluation Algorithm (Python Version)

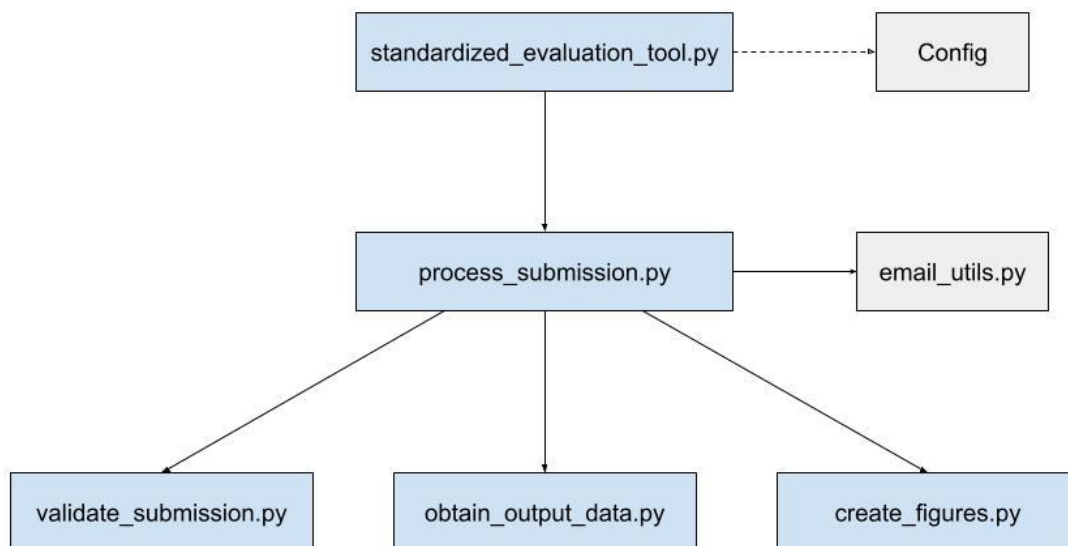


Figure 6.1 - Heropage of an authenticated user

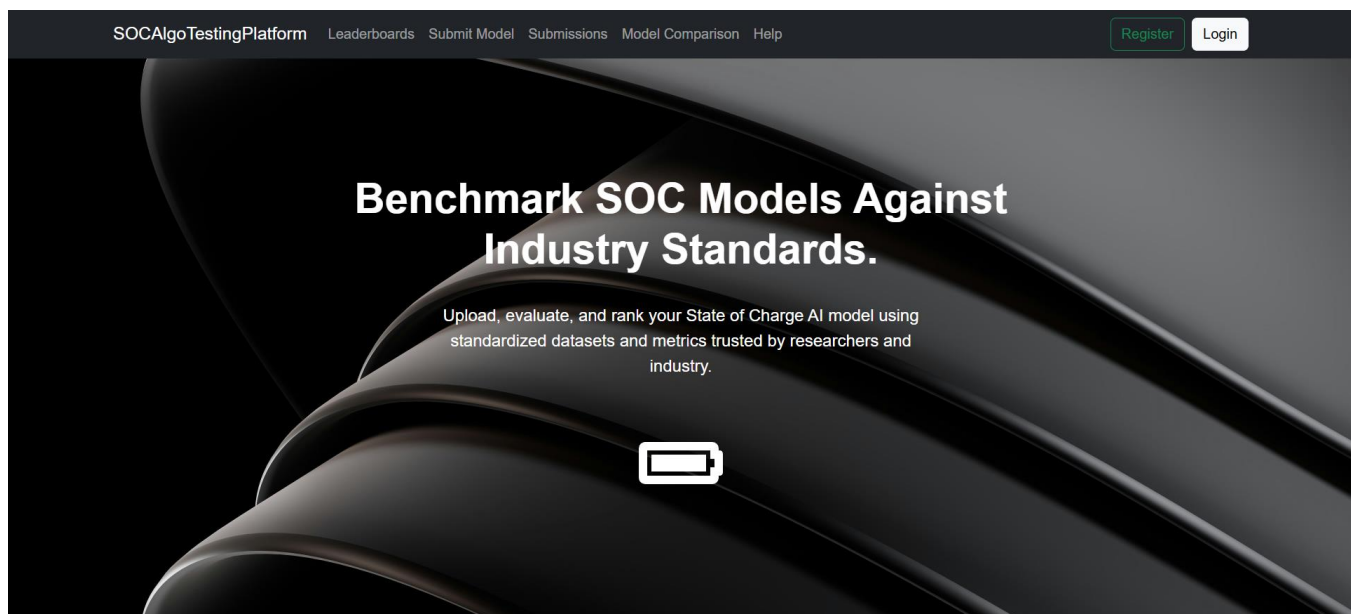


Figure 6.2 - Registration page

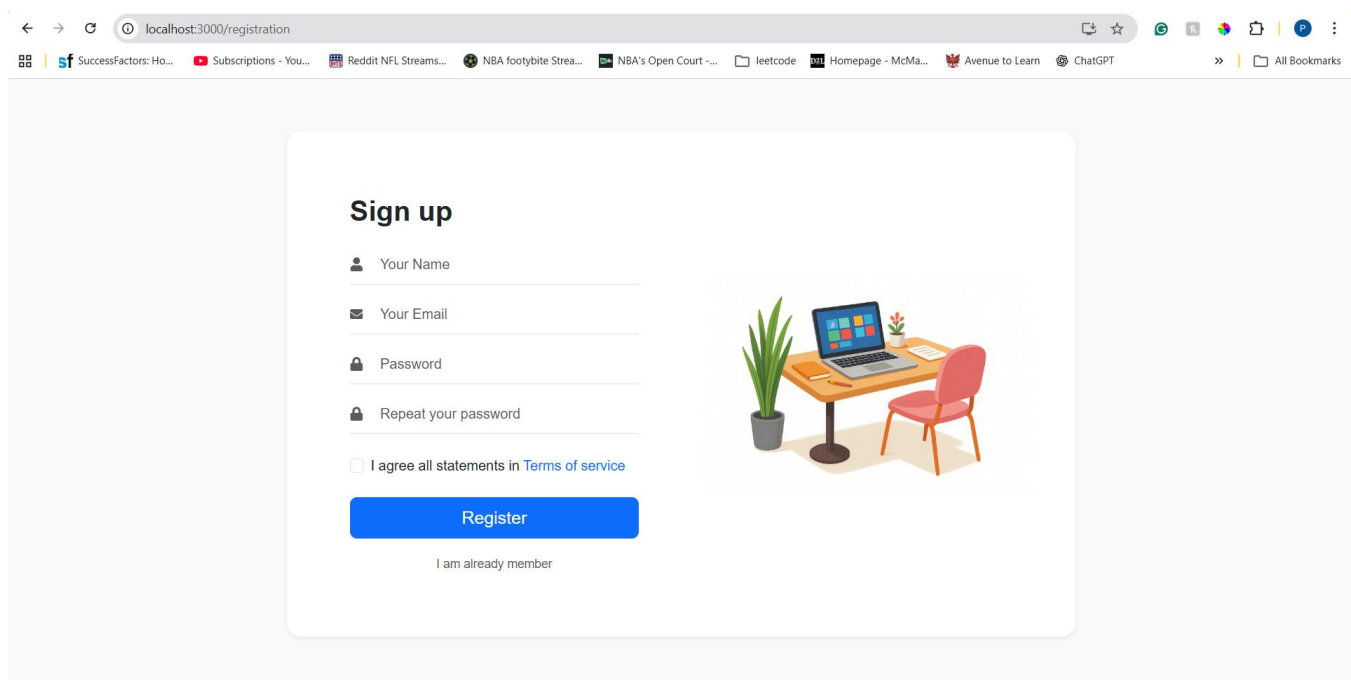


Figure 6.3 - Model Submissions page

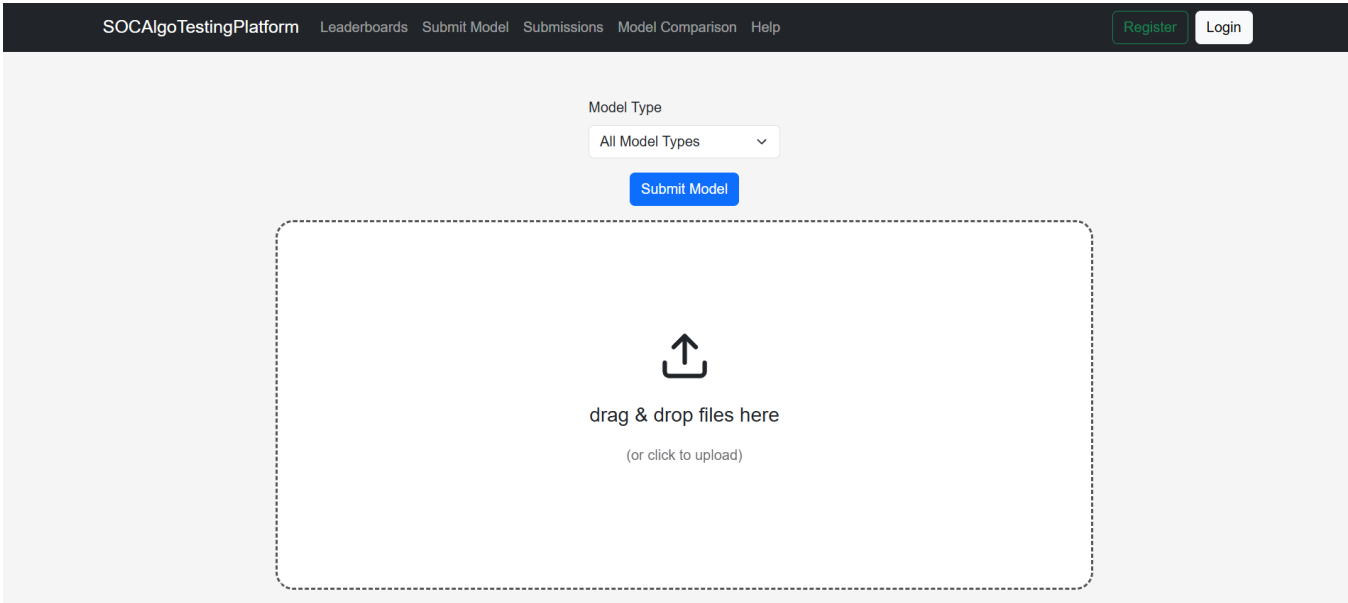


Figure 6.4 - Leaderboards Page

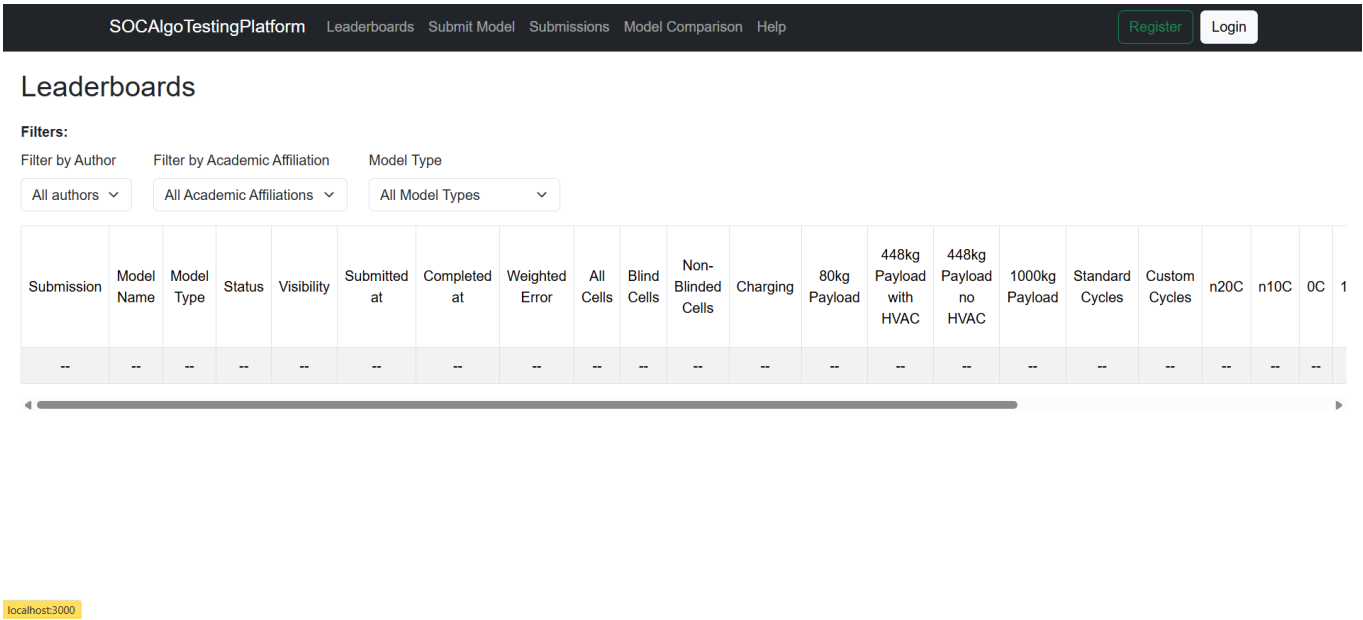


Figure 6.5 - Submission Page

Submissions

Filters:

Model Type

All Model Types

Submission	Model Name	Model Type	Status	Visibility	Submitted at	Completed at	Weighted Error	All Cells	Blind Cells	Non-Blinded Cells	Charging	80kg Payload	448kg Payload with HVAC	448kg Payload no HVAC	1000kg Payload	Standard Cycles	Custom Cycles	n20C	n10C	0C	1
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Figure 6.6 - Model Comparison Page

Model Comparison

Display Graphical Comparison

Model 1

Filters:

Filter by Author

Filter by Academic Affiliation

Model Type

All authors

All Academic Affiliations

All Model Types

Ranking	Submission	Author	Affiliation	Model Name	Model Type	Weighted Error
--	--	--	--	--	--	--

Model 2

Filters:

Filter by Author

Filter by Academic Affiliation

Model Type

All authors

All Academic Affiliations

All Model Types

Ranking	Submission	Author	Affiliation	Model Name	Model Type	Weighted Error
--	--	--	--	--	--	--